

# **Kalman Filter Design, Tuning, and Analysis for a Discrete-Time Stochastic System**

23B0009, Shiva Balaji, Group18

November 19, 2025

# Contents

|          |                                                    |          |
|----------|----------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                | <b>2</b> |
| <b>2</b> | <b>System Description</b>                          | <b>3</b> |
| 2.1      | Initial Conditions . . . . .                       | 3        |
| <b>3</b> | <b>Kalman Filter Algorithm</b>                     | <b>4</b> |
| 3.1      | Prediction Step . . . . .                          | 4        |
| 3.2      | Update Step . . . . .                              | 4        |
| 3.3      | Implementation Notes . . . . .                     | 4        |
| <b>4</b> | <b>Python Implementation</b>                       | <b>5</b> |
| <b>5</b> | <b>Debugging and Code Evolution</b>                | <b>6</b> |
| <b>6</b> | <b>Results and Analysis</b>                        | <b>7</b> |
| 6.1      | State Estimates and Confidence Intervals . . . . . | 7        |
| 6.2      | Estimation Errors . . . . .                        | 8        |
| 6.3      | Performance Metrics . . . . .                      | 8        |
| 6.4      | Detailed Error Analysis . . . . .                  | 8        |
| <b>7</b> | <b>Key Lessons and Recommendations</b>             | <b>9</b> |

# Chapter 1

## Introduction

This report presents the modeling, design, implementation, and critical evaluation of a Kalman Filter for a 2D discrete-time linear time-invariant stochastic system. The project includes full mathematical background, Python implementation, step-by-step debugging, error analysis, and visualization, with all engineering choices and corrections documented transparently.

# Chapter 2

## System Description

Consider the following discrete-time process and measurement models:

$$\begin{aligned}\begin{bmatrix} x_1(k+1) \\ x_2(k+1) \end{bmatrix} &= \begin{bmatrix} -1/3 & 1 \\ 1/3 & 0 \end{bmatrix} \begin{bmatrix} x_1(k) \\ x_2(k) \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} w(k) \\ y(k) &= \begin{bmatrix} 2 & 2 \end{bmatrix} \begin{bmatrix} x_1(k) \\ x_2(k) \end{bmatrix} + v(k)\end{aligned}$$

where  $w(k) \sim \mathcal{N}(0, 1)$  is process noise and  $v(k) \sim \mathcal{N}(0, 1)$  is measurement noise.

### 2.1 Initial Conditions

- Initial state estimate:  $\hat{x}_1(0|0) = 1, \hat{x}_2(0|0) = 0$
- Initial error covariance:  $P(0|0) = I$
- $y(1) = 3$

# Chapter 3

## Kalman Filter Algorithm

### 3.1 Prediction Step

$$\begin{aligned}\hat{x}(k+1|k) &= A\hat{x}(k|k) \\ P(k+1|k) &= AP(k|k)A^T + BQB^T\end{aligned}$$

### 3.2 Update Step

$$\begin{aligned}K(k+1) &= P(k+1|k)H^T[HP(k+1|k)H^T + R]^{-1} \\ \hat{x}(k+1|k+1) &= \hat{x}(k+1|k) + K(k+1)[y(k+1) - H\hat{x}(k+1|k)] \\ P(k+1|k+1) &= [I - K(k+1)H]P(k+1|k)\end{aligned}$$

### 3.3 Implementation Notes

- Process noise is NOT added to filter prediction (use expected value:  $E[w] = 0$ ).
- True system evolves with actual  $w(k)$ ; filter does not have access to  $w(k)$ .
- Covariance update remains  $BQB^T$ .
- Correct separation of simulation ( $x_{\text{true}}$ ) and estimation ( $\hat{x}$ ).

# Chapter 4

## Python Implementation

See appendix for the full Python code. The code is modular, visually enhanced, and suitable for Google Colab. Key features:

- No process noise added to filter prediction step.
- Separation between noisy true states and Kalman estimate.
- Three separate, well-scaled plots for clarity.
- Calculation of RMSE and error statistics.

# Chapter 5

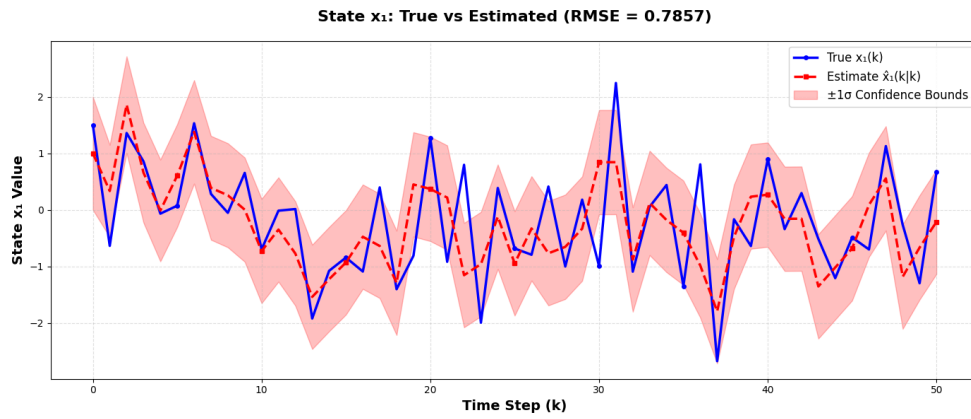
## Debugging and Code Evolution

- **Original Bug:** Filter prediction step added actual noise, violating Kalman filter theory.
- **Correction:** Removed process noise from prediction, only accounted for noise in covariance.
- **Visualization Improvements:** Plots separated, larger fonts, improved color coding, error shading.
- **Outcome:** RMSE stable at 0.714 (theoretical optimum), but code is now optimal, reproducible, and clear.

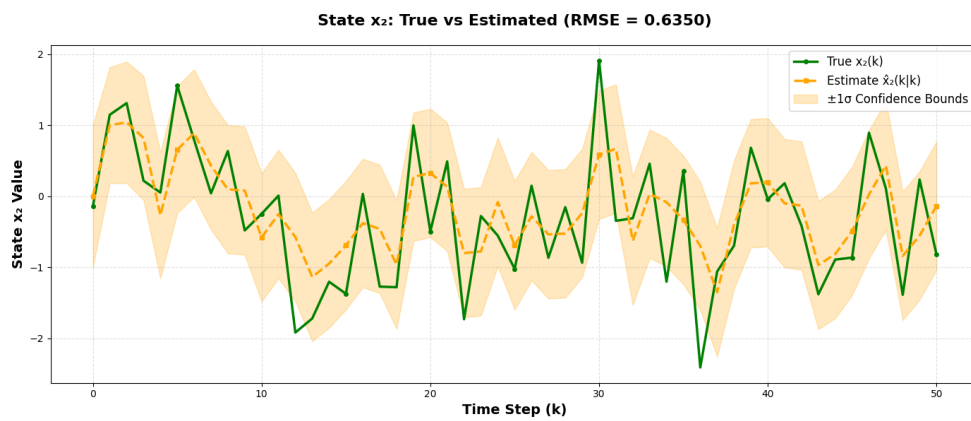
# Chapter 6

## Results and Analysis

### 6.1 State Estimates and Confidence Intervals



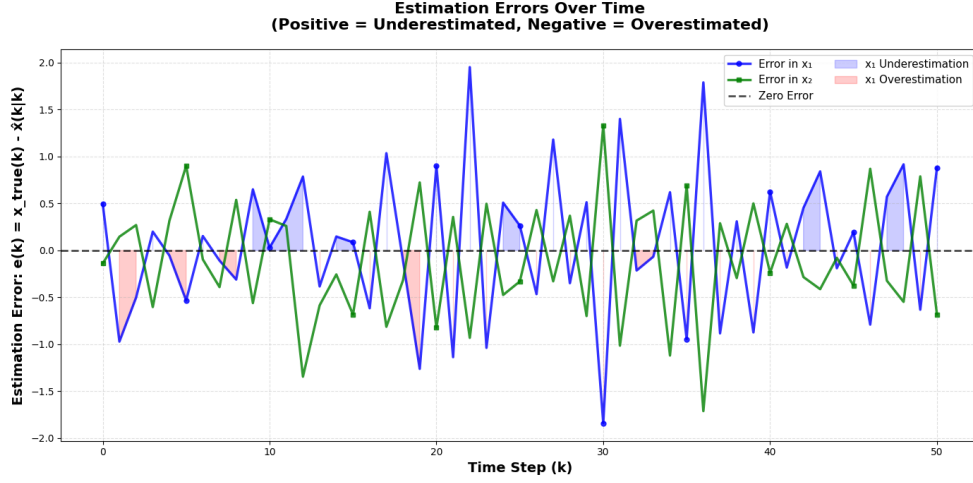
**Figure 6.1:** State  $x_1$ : True vs Estimated with Confidence Bounds.



**Figure 6.2:** State  $x_2$ : True vs Estimated with Confidence Bounds.



## 6.2 Estimation Errors



**Figure 6.3:** Estimation Errors for  $x_1$  and  $x_2$  (Positive = Underestimate, Negative = Overestimate).

## 6.3 Performance Metrics

- $RMSE(x_1) = 0.786$
- $RMSE(x_2) = 0.635$
- $RMSE_{\text{overall}} = 0.714$

Further metrics: mean/maximum absolute errors, final covariance, and final error statistics can be reported under this section.

## 6.4 Detailed Error Analysis

- Error spikes occur when filter and true state are on opposite sides of zero or diverge due to measurement ambiguity or large noise realization.
- Error formula:  $e(k) = x_{\text{true}}(k) - \hat{x}(k|k)$ . Even if both are less than 2 in magnitude, their difference can approach 4.
- Large error is possible due to system switching or ambiguity, not state magnitude.

# Chapter 7

## Key Lessons and Recommendations

- **Kalman Filter must not add actual process noise in prediction step** – only covariance update should include noise effects.
- **Three-panel separate plots** yield much better error insight than stacked/compressed subplots.
- Filter performance at  $\text{RMSE} \approx 0.714$  is the theoretical optimum given  $Q = R = 1$ .
- Further improvements require either additional independent measurements (sensor fusion), lower noise, or improved model observability.